STUDENT LABORATORY MANUAL

Module 5: Introduction to Computer Graphics and Visual Computing

20 Practice Exercises

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 5 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + GLEW + FreeGLUT

Name	SERRANO, LIAM BRENT C.
Section	TN35
Github	https://github.com/TGOLiam/opengl-exercises

How to Use This Manual

This manual contains the practice exercises and answer key:

- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Each exercise gives you an Objective, a Task question describing exactly what your program must do, and Hints to point you in the right direction. Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

This module requires GLEW in addition to FreeGLUT. On Windows/MSYS2 UCRT64, install it with: pacman -S mingw-w64-ucrt-x86_64-glew. Include <GL/glew.h> BEFORE <GL/glut.h>, and call glewInit() after glutCreateWindow()/glutDisplayFunc() but before glutMainLoop(). All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__  #include
<GLUT/glut.h> #else
#include    <GL/glew.h>
#include     <GL/glut.h>
#endif
#include     <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with g++ against GLEW and FreeGLUT.

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard. Each exercise's "Task" line is the exact question you must answer with your program.

Easy Exercises

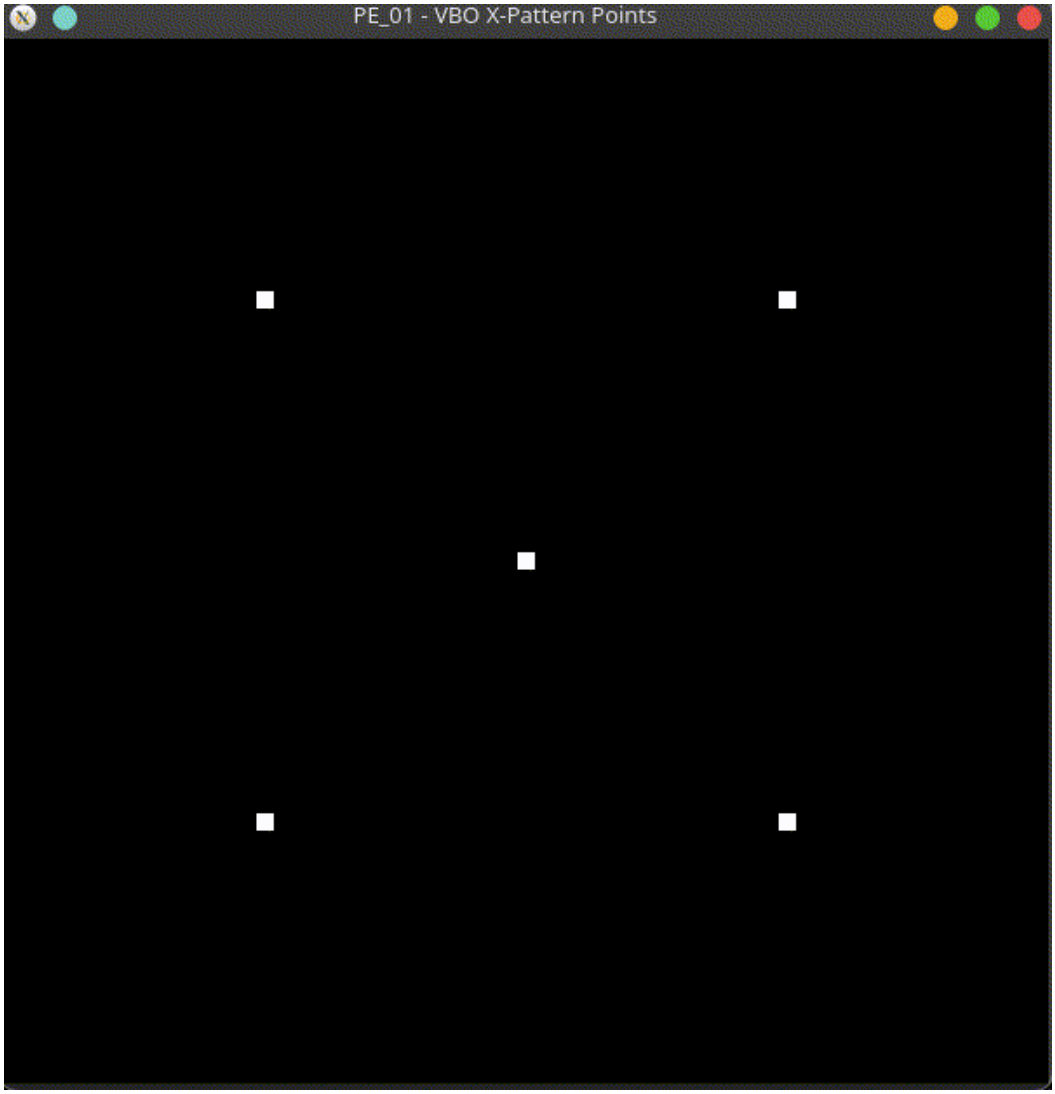
Exercise Q01 — VBO X-Pattern Points [Easy]

Objective: Practice converting GL_POINTS to a VBO using the 5-step pattern.

Task (Question): Convert an immediate-mode GL_POINTS block (5 points forming an X pattern with a center point) into a VBO, using all 5 steps (generate, bind, upload, point-and-draw with an offset, clean up).

Hints:

- Set glPointSize BEFORE glDrawArrays, just as you would before glBegin.
- Once a buffer is bound, the last argument to glVertexPointer is a byte OFFSET (usually 0), not a real pointer.



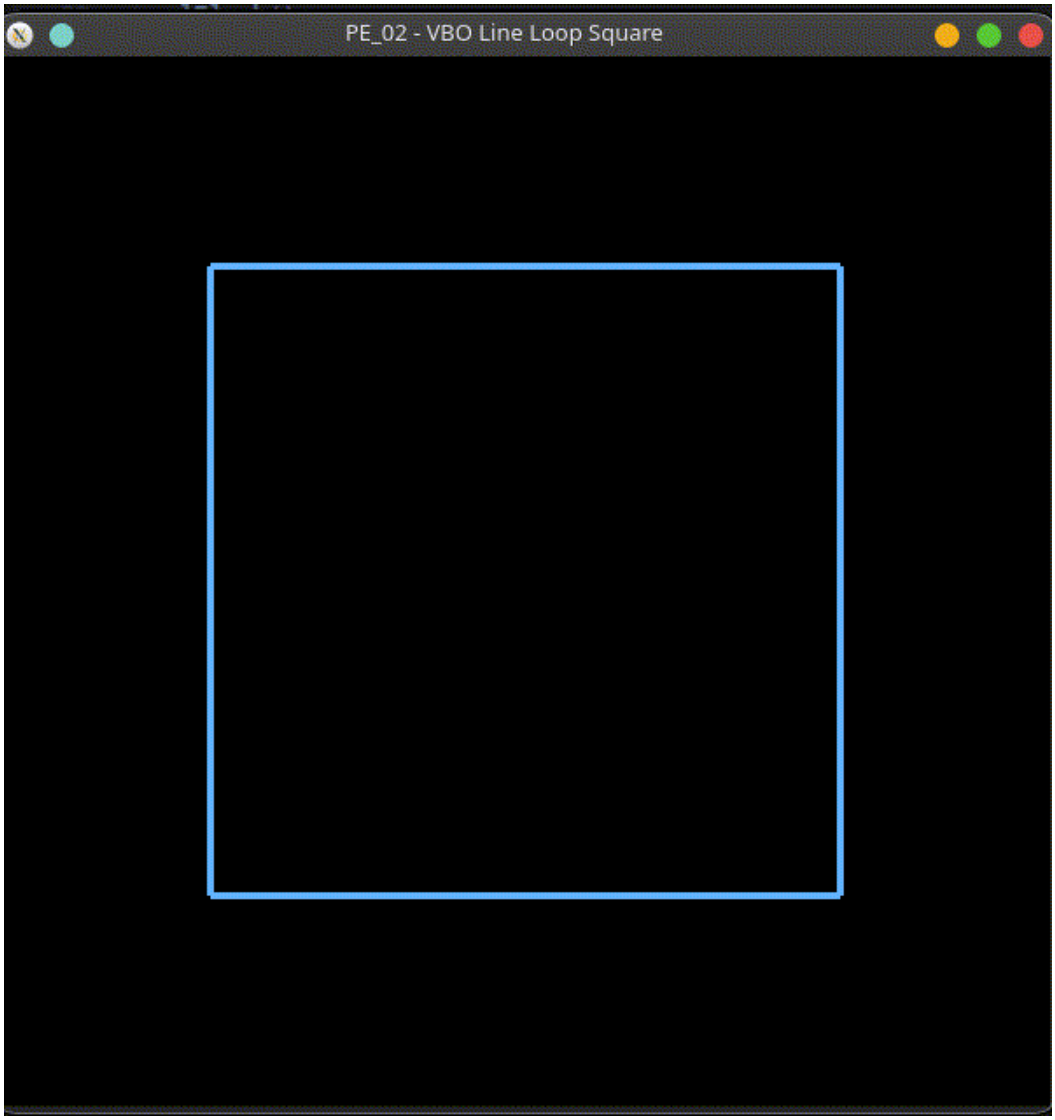
Exercise Q02 — VBO Line Loop Square [Easy]

Objective: Practice GL_LINE_LOOP sourced from a VBO instead of GL_QUADS.

Task (Question): Draw the outline of a square using GL_LINE_LOOP via a VBO, with 4 unique vertices of your own choosing.

Hints:

- GL_LINE_LOOP automatically connects the last vertex back to the first — you do not need to repeat the first vertex.
- glLineWidth (called before the draw) makes the outline easier to see.



Exercise Q03 — Static Hexagon With the Correct Usage Hint [Easy]

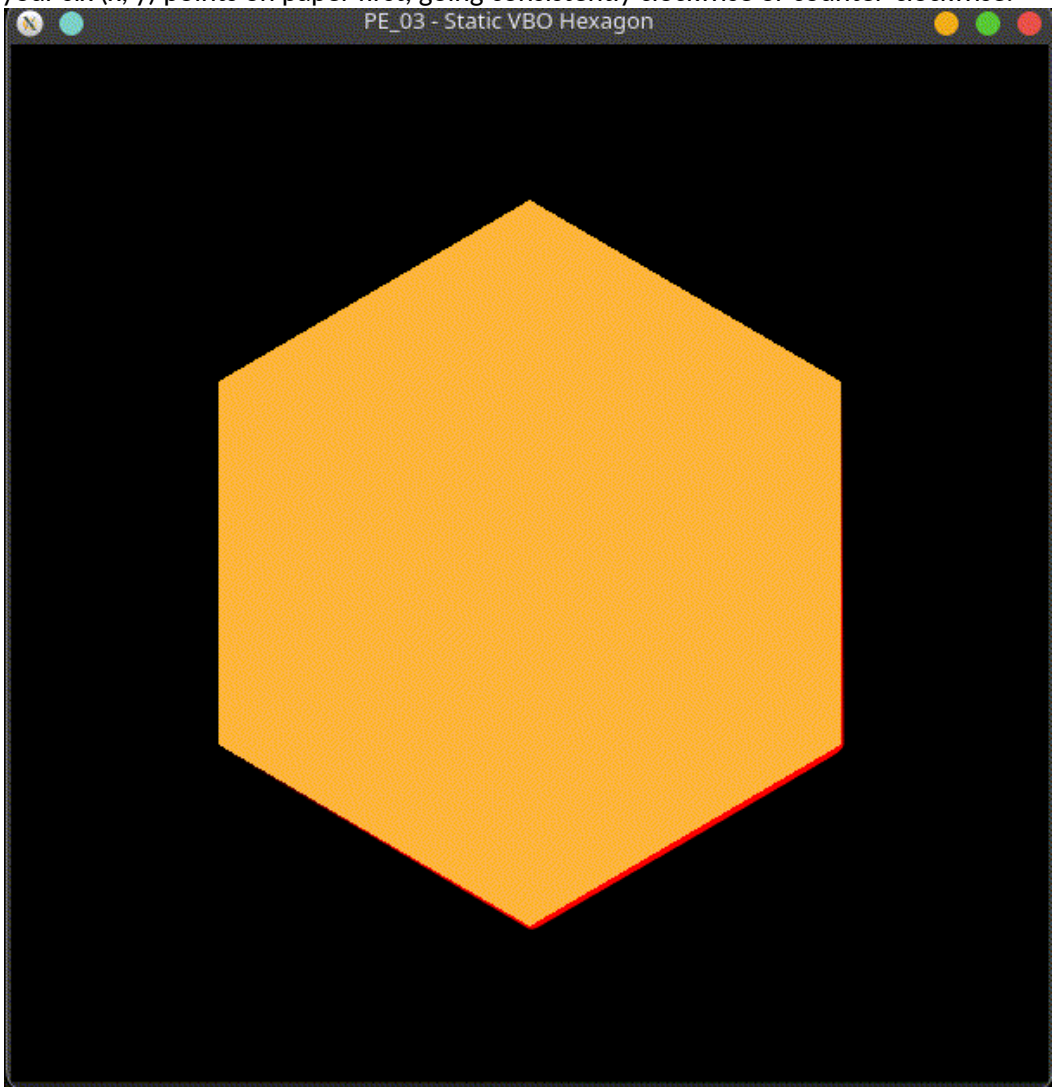
Objective: Practice choosing and justifying a glBufferData usage hint for data that never changes.

Task (Question): Draw a filled, six-sided hexagon using GL_POLYGON via a VBO, choosing the usage hint that best matches data uploaded once and drawn many times, and add a code comment explaining WHY you chose it.

Hints:

- Data set once and used many times without modification matches GL_STATIC_DRAW.

- Plan your six (x, y) points on paper first, going consistently clockwise or counter-clockwise.



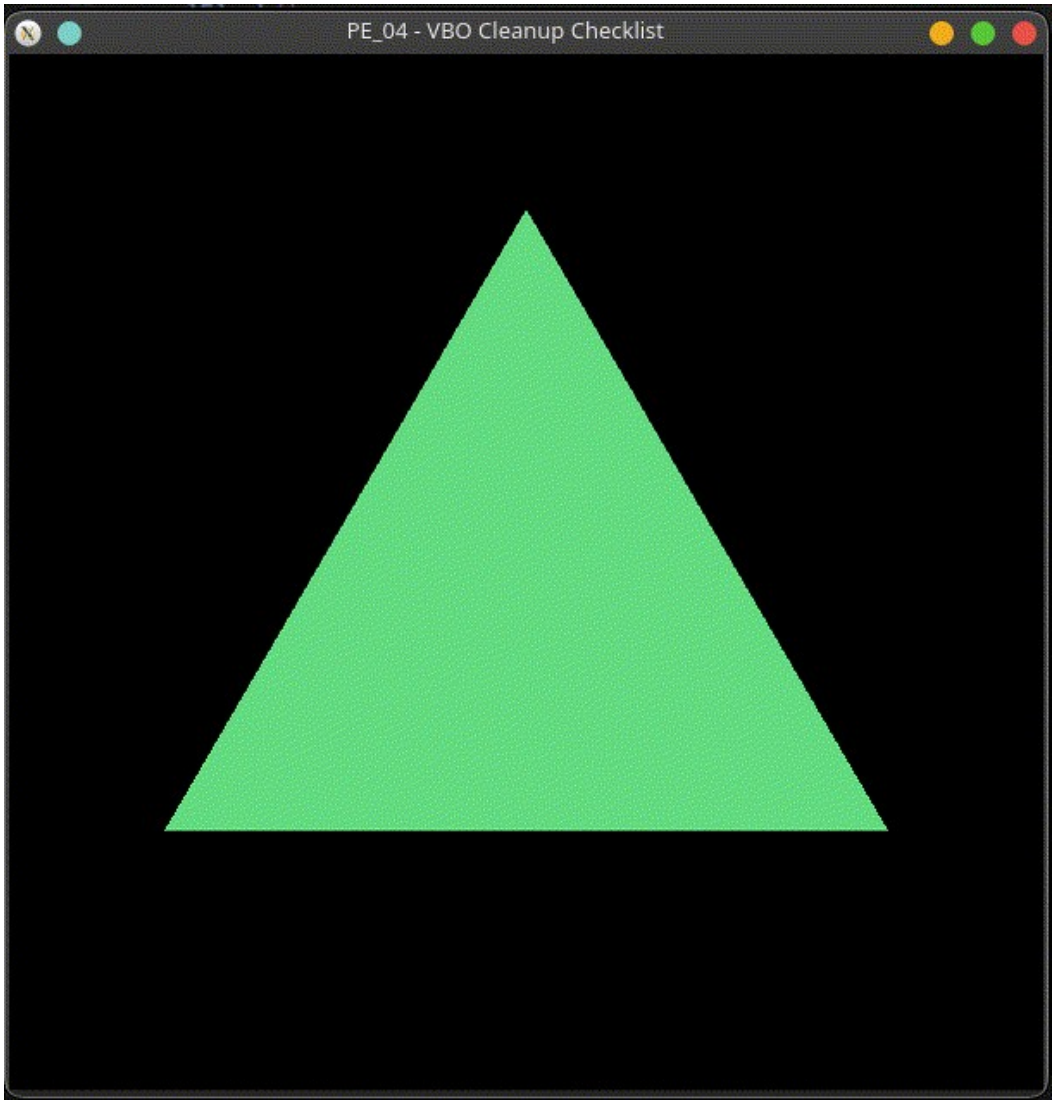
Exercise Q04 — Full Cleanup Checklist [Easy]

Objective: Practice the complete VBO cleanup sequence (disable, unbind, delete).

Task (Question): Draw any simple shape from a VBO, and BEFORE the program exits, perform the full cleanup checklist: glDisableClientState, glBindBuffer(target, 0), and glDeleteBuffers, with a comment above each line explaining what it undoes.

Hints:

- Unbinding the buffer (binding 0) should happen before deleting it.
- glDeleteBuffers takes the ADDRESS of your GLuint ID variable, the same as glGenBuffers.



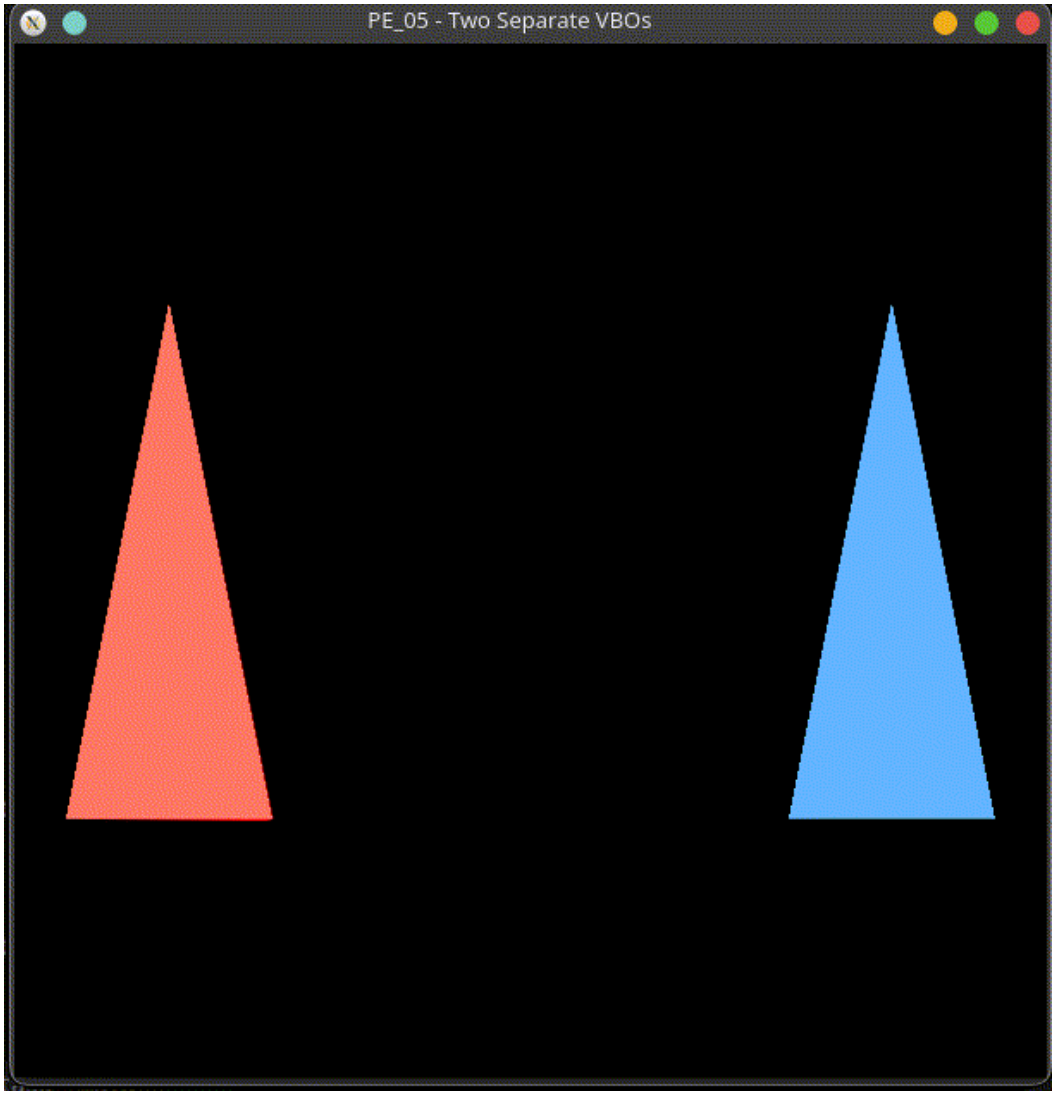
Exercise Q05 — Two Separate VBOs Side by Side [Easy]

Objective: Practice managing two independent GLuint buffer IDs in the same program.

Task (Question): Draw two triangles side by side, each one sourced from its OWN separate VBO (two GLuint IDs, two glGenBuffers/glBindBuffer/glBufferData sequences).

Hints:

- Give the two GLuint variables clearly different names so you don't mix up which buffer is bound when.
- Re-bind the correct buffer immediately before each glDrawArrays call.



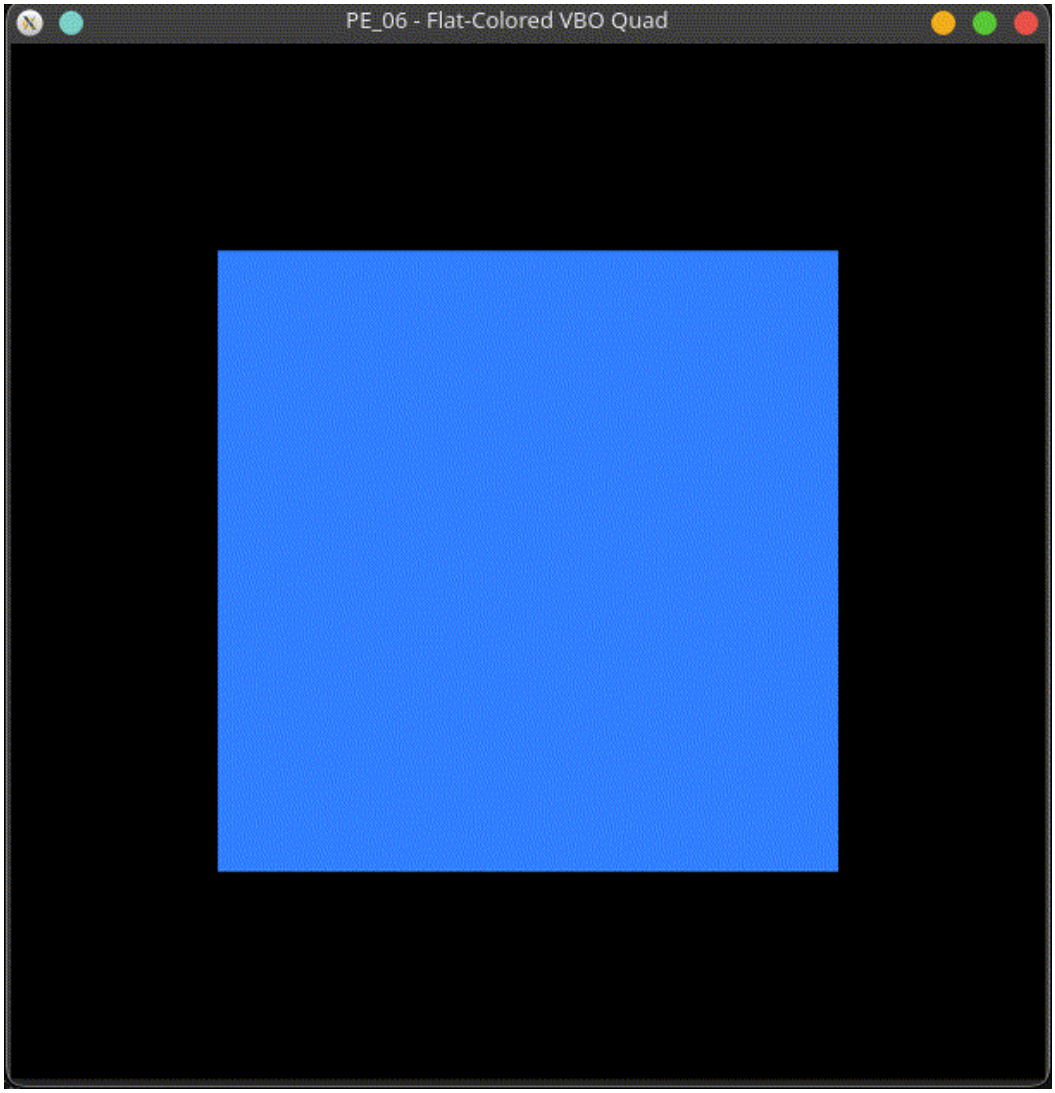
Exercise Q06 — Flat-Colored VBO Quad [Easy]

Objective: Practice a basic VBO conversion for GL_QUADS with a single flat color.

Task (Question): Convert an immediate-mode filled square into a VBO using GL_QUADS, keeping a single flat glColor3f color for the whole shape (no color array needed yet).

Hints:

- You only need ONE VBO here — position data only.
- Call glColor3f once, before the draw call, exactly as you would in immediate mode.



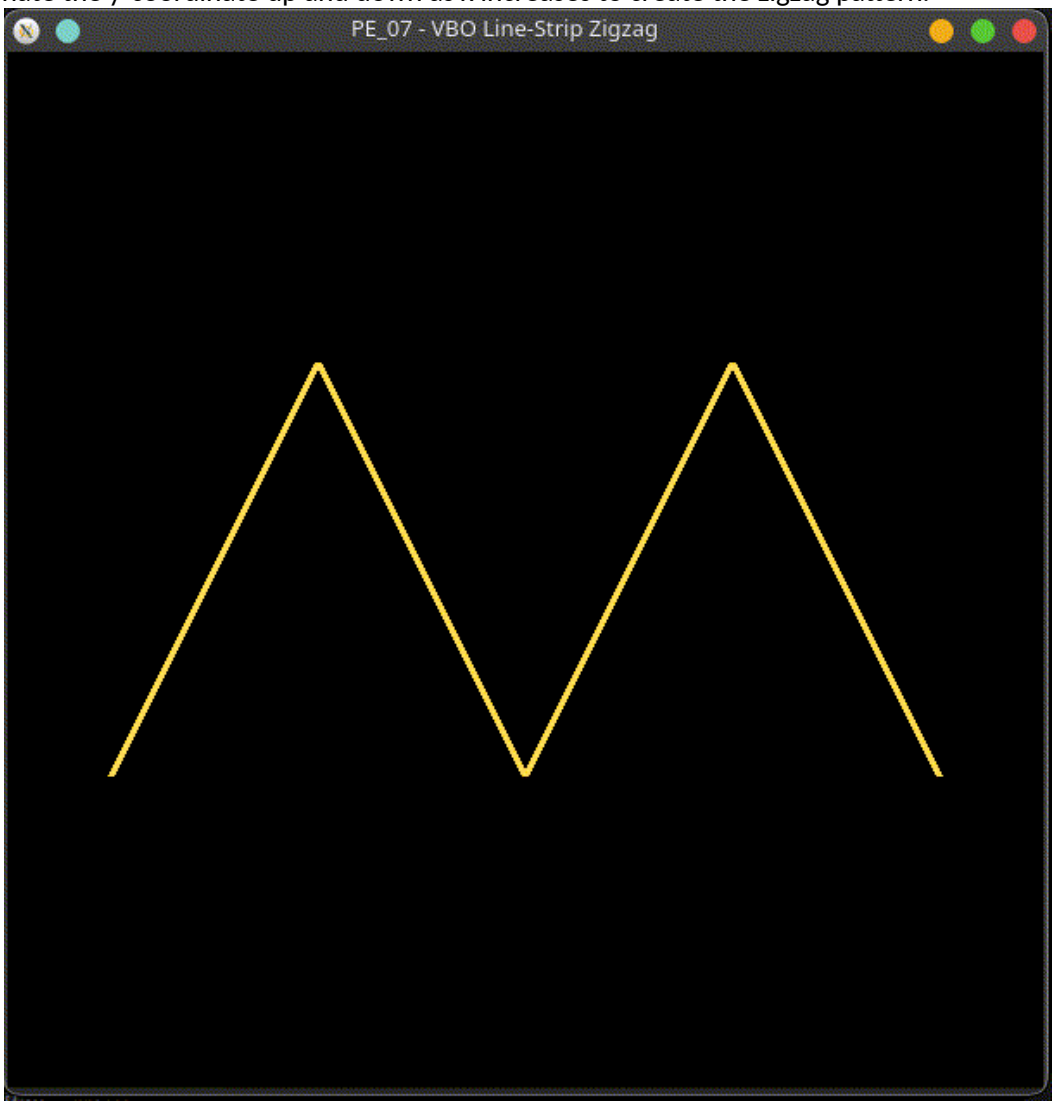
Exercise Q07 — VBO Line-Strip Zigzag [Easy]

Objective: Practice GL_LINE_STRIP sourced from a VBO with more than 2 vertices.

Task (Question): Draw a zigzag line (at least 5 vertices) using GL_LINE_STRIP via a VBO.

Hints:

- GL_LINE_STRIP connects each vertex to the next, but does NOT close back to the first (unlike GL_LINE_LOOP).
- Alternate the y-coordinate up and down as x increases to create the zigzag pattern.



Medium Exercises

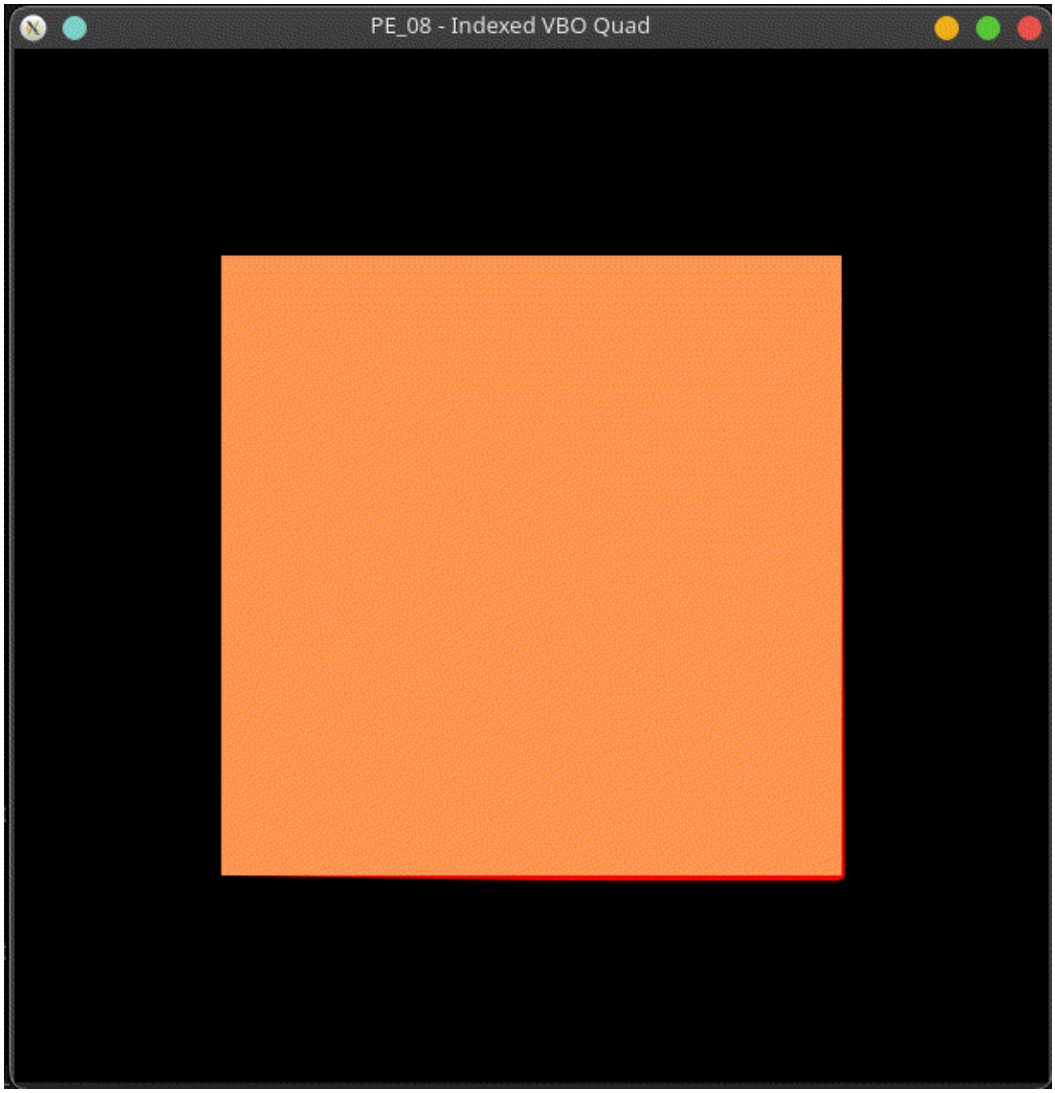
Exercise Q08 — Indexed Quad: Position VBO + Separate Index VBO [Medium]

Objective: Practice combining a position VBO with a GL_ELEMENT_ARRAY_BUFFER index VBO.

Task (Question): Build a quad from 4 unique GPU-resident vertices plus a 6-entry GPU-resident index buffer, drawn with glDrawElements.

Hints:

- You need TWO buffer IDs: one bound to GL_ARRAY_BUFFER, one bound to GL_ELEMENT_ARRAY_BUFFER.
- The index array's data type (GLubyte, GLuint, etc.) must match the `type` argument you pass to glDrawElements.



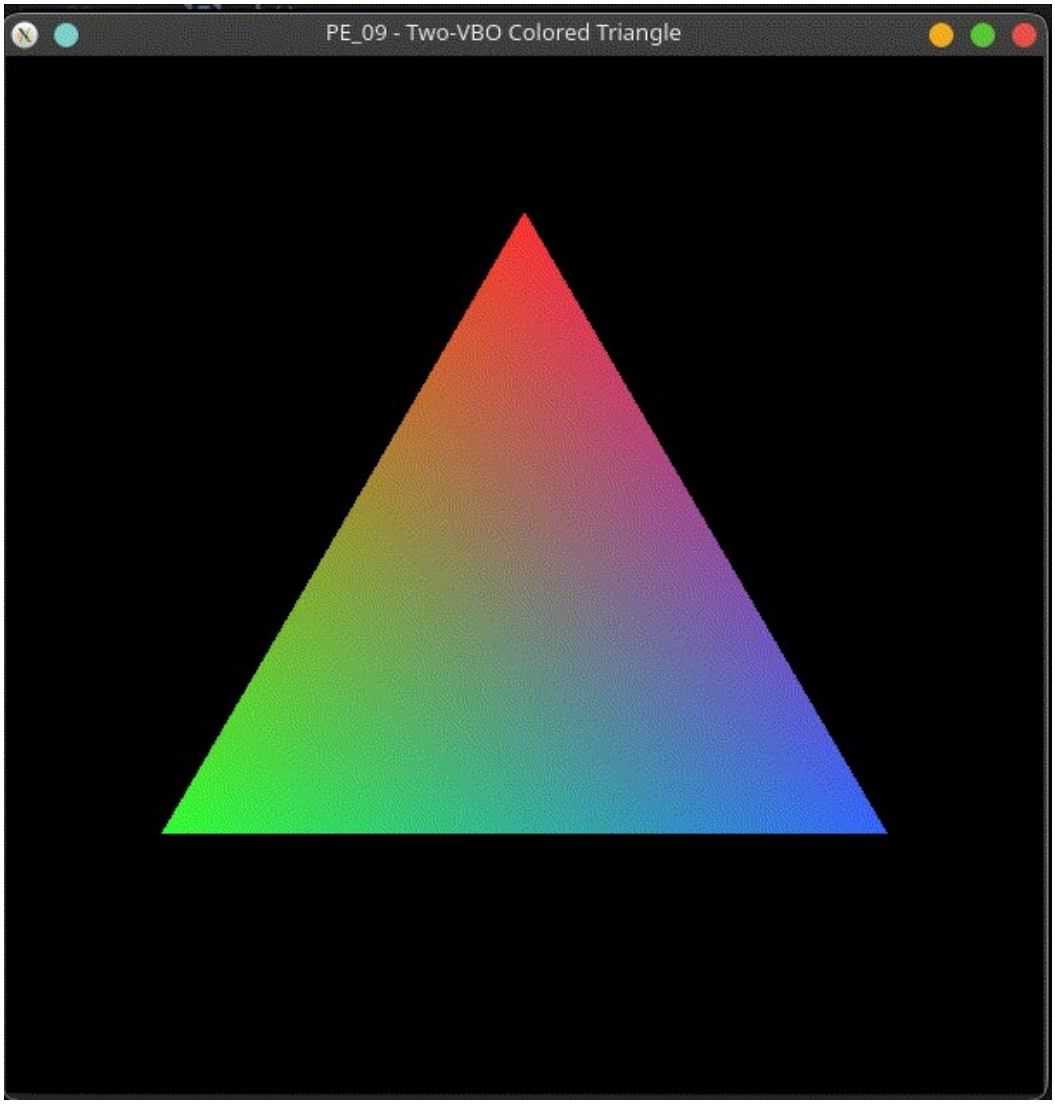
Exercise Q09 — Two-VBO Colored Triangle [Medium]

Objective: Practice combining a position VBO and a SEPARATE color VBO (not interleaved).

Task (Question): Draw a triangle with a different color at each vertex, using a position VBO and a separate color VBO (two independent buffers, not one interleaved buffer).

Hints:

- Both GL_VERTEX_ARRAY and GL_COLOR_ARRAY must be enabled, and each needs its own buffer bound right before its pointer call.
- Bind the position buffer and call glVertexPointer, THEN bind the color buffer and call glColorPointer.



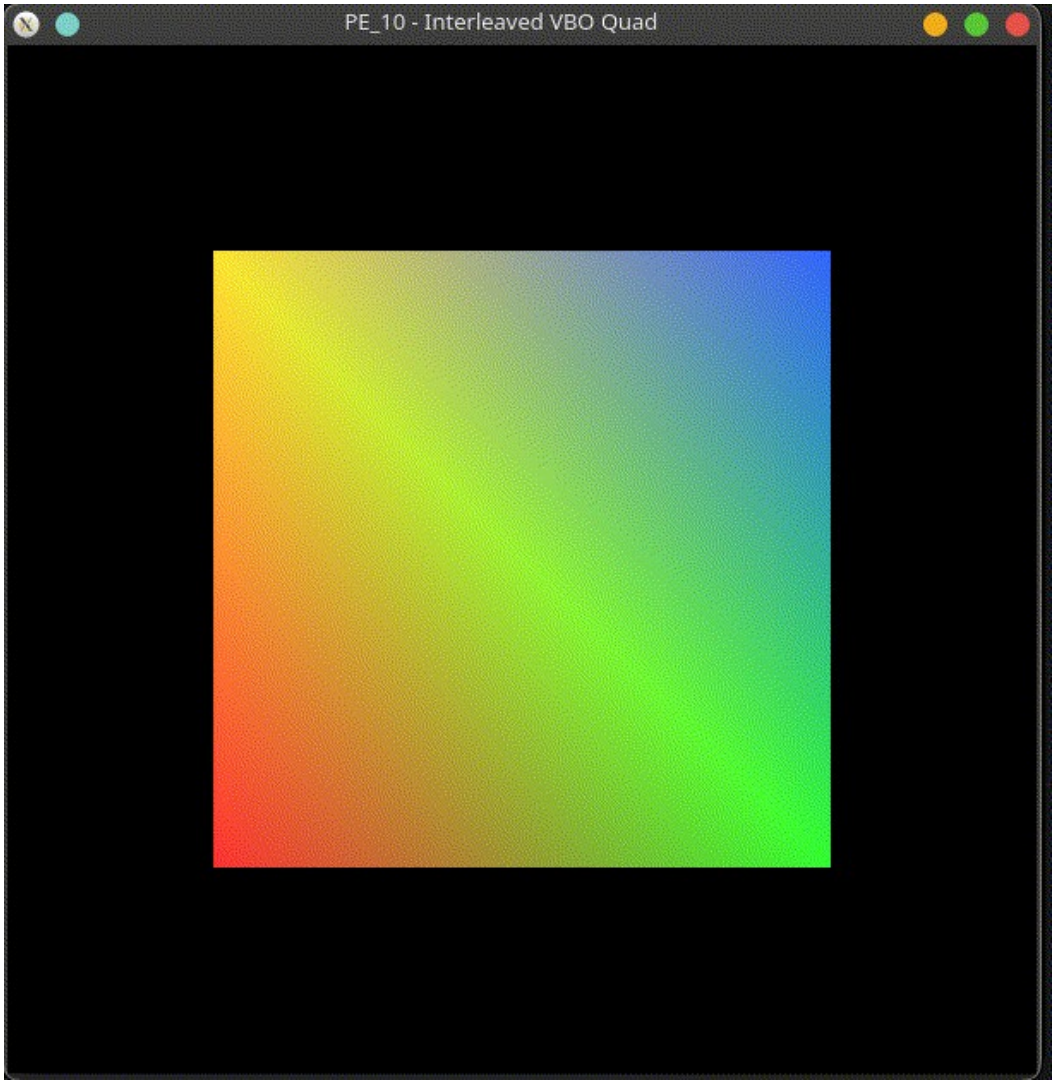
Exercise Q10 — Interleaved VBO Quad [Medium]

Objective: Practice building and reading an interleaved position+color VBO with a non-zero stride.

Task (Question): Build a single interleaved VBO holding both position and color data for a quad, and render it using stride-based glVertexPointer/glColorPointer calls.

Hints:

- Each vertex's 6 floats are x,y,z,r,g,b in that order — stride is 6 * sizeof(GLfloat).
- The color pointer's byte-offset argument must skip past the 3 position floats: (void*)(3 * sizeof(GLfloat)).



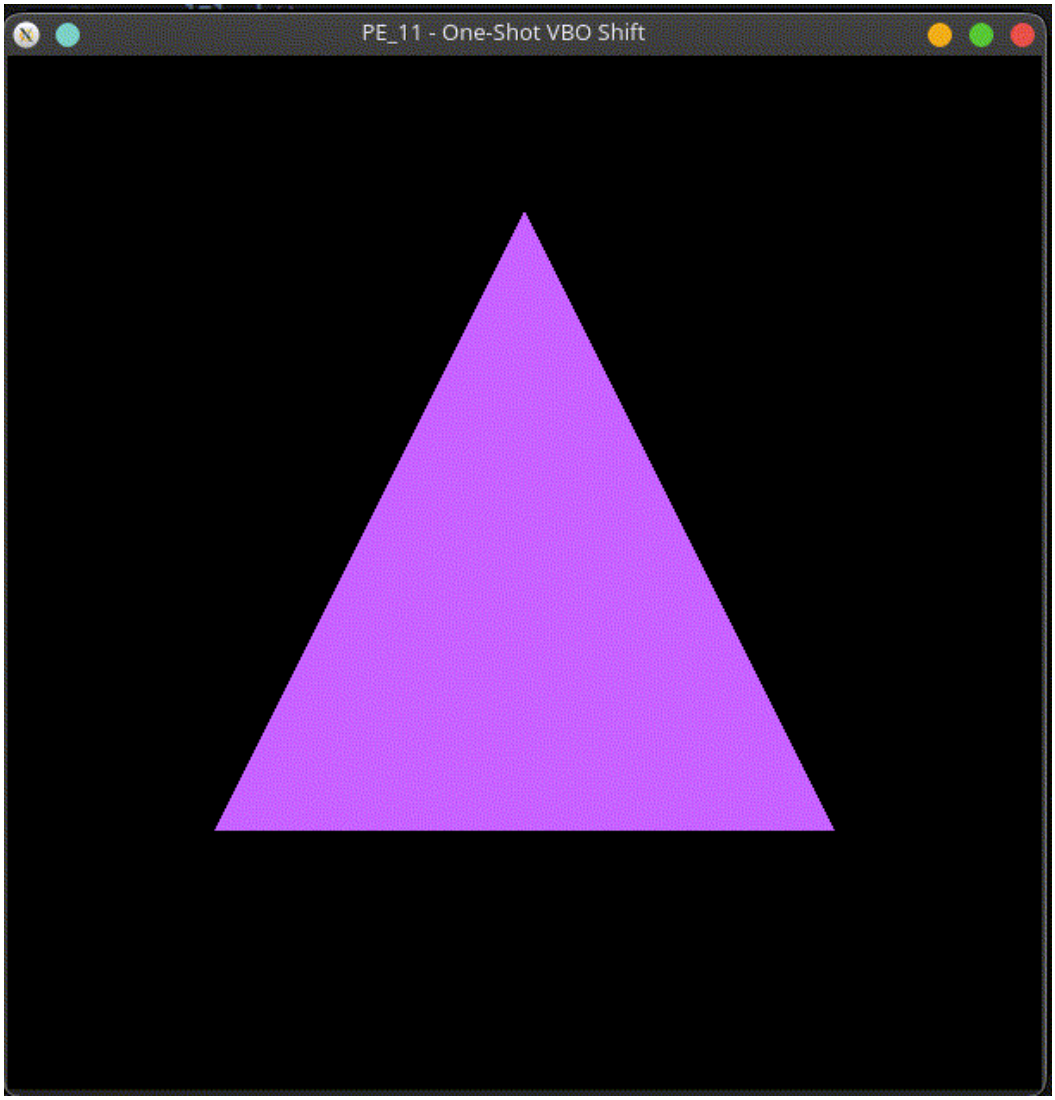
Exercise Q11 — One-Shot glMapBuffer Vertical Shift [Medium]

Objective: Practice mapping a VBO, editing it once, and unmapping it before drawing.

Task (Question): Draw a triangle from a VBO, then use glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE) to shift ONE vertex upward by editing the mapped memory directly, calling glUnmapBuffer before the draw call.

Hints:

- glMapBuffer returns a void* — cast it to GLfloat* before indexing into it.
- ALWAYS call glUnmapBuffer immediately after you finish editing, before any draw call uses that buffer.



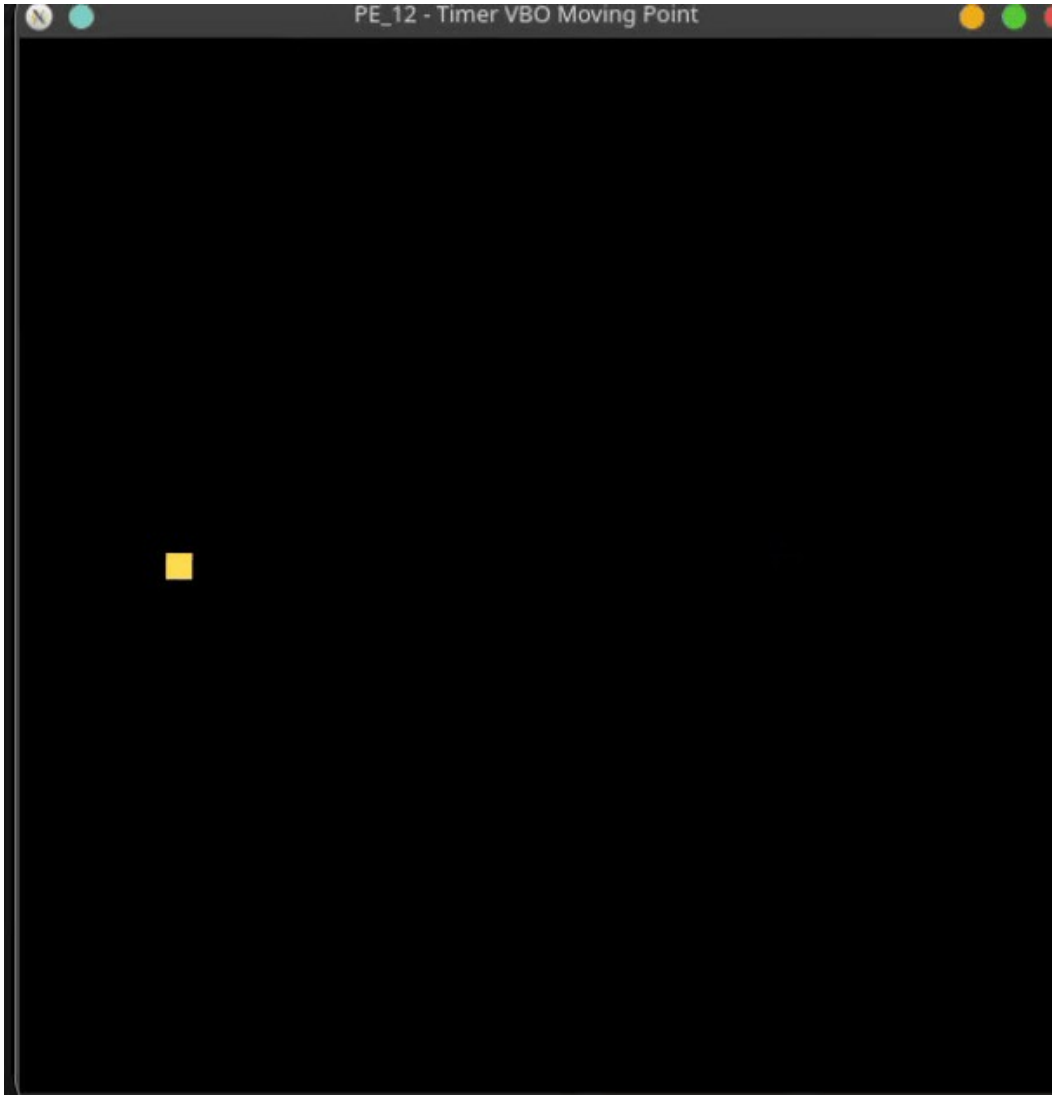
Exercise Q12 — Timer + glMapBuffer Moving Point [Medium]

Objective: Practice a glutTimerFunc callback that repeatedly maps and edits a VBO.

Task (Question): Animate a single GL_POINTS vertex so it moves one step to the right every second, using a glutTimerFunc callback that maps the VBO, updates the x-coordinate, unmaps it, and reschedules itself.

Hints:

- Call glutPostRedisplay() at the end of your timer callback so the change actually appears on screen.
- Reschedule the timer by calling glutTimerFunc again as the LAST line inside the callback itself.



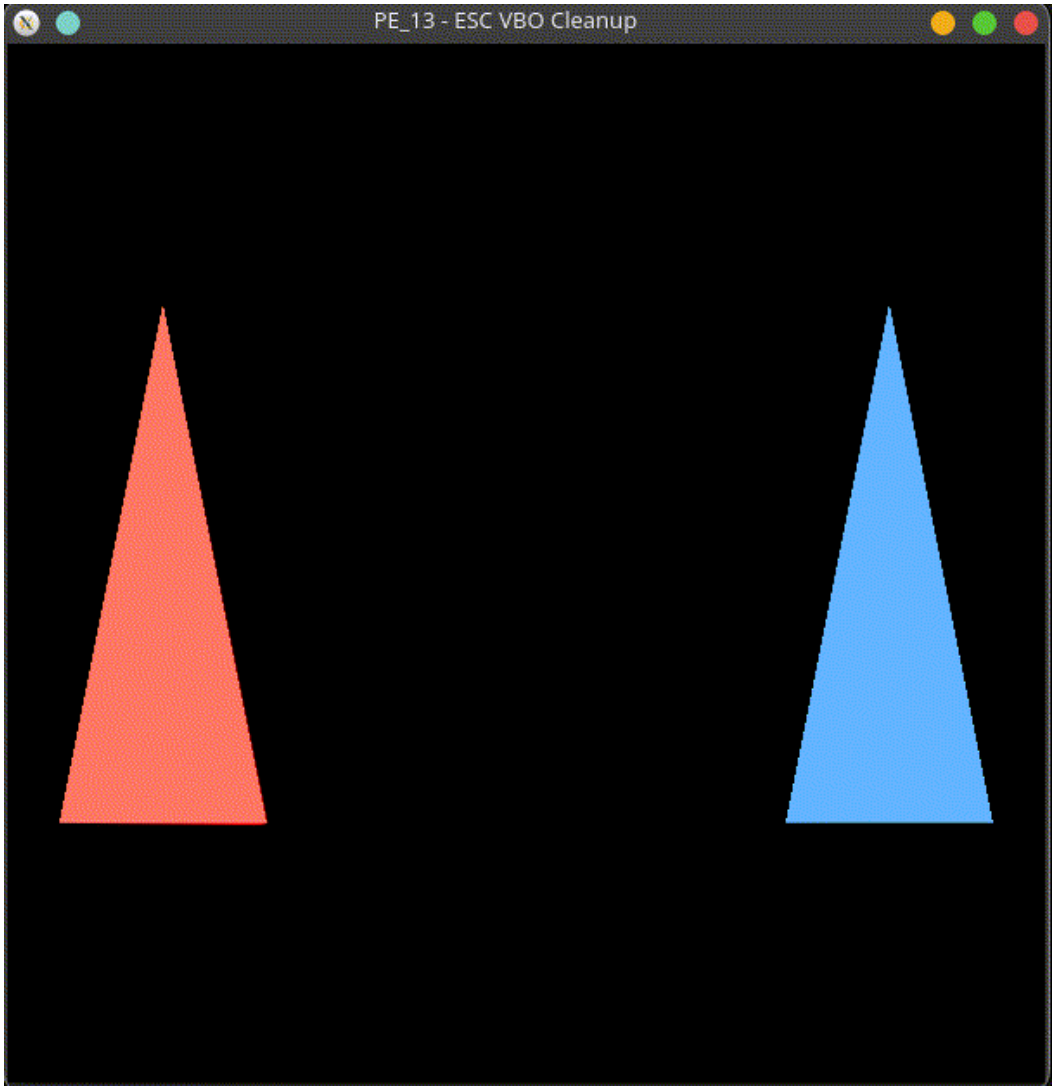
Exercise Q13 — ESC Cleanup Releasing Two VBOs [Medium]

Objective: Practice cleanup discipline when a program owns MORE than one buffer.

Task (Question): Draw a scene using two separate VBOs, and write a monitorKeyboard() handler that releases BOTH buffers with glDeleteBuffers before exit(0) when ESC is pressed.

Hints:

- Delete each buffer by its own GLuint ID — one glDeleteBuffers call per buffer, or one call with an array of both IDs.
- Register your keyboard handler with glutKeyboardFunc inside main(), before glutMainLoop().



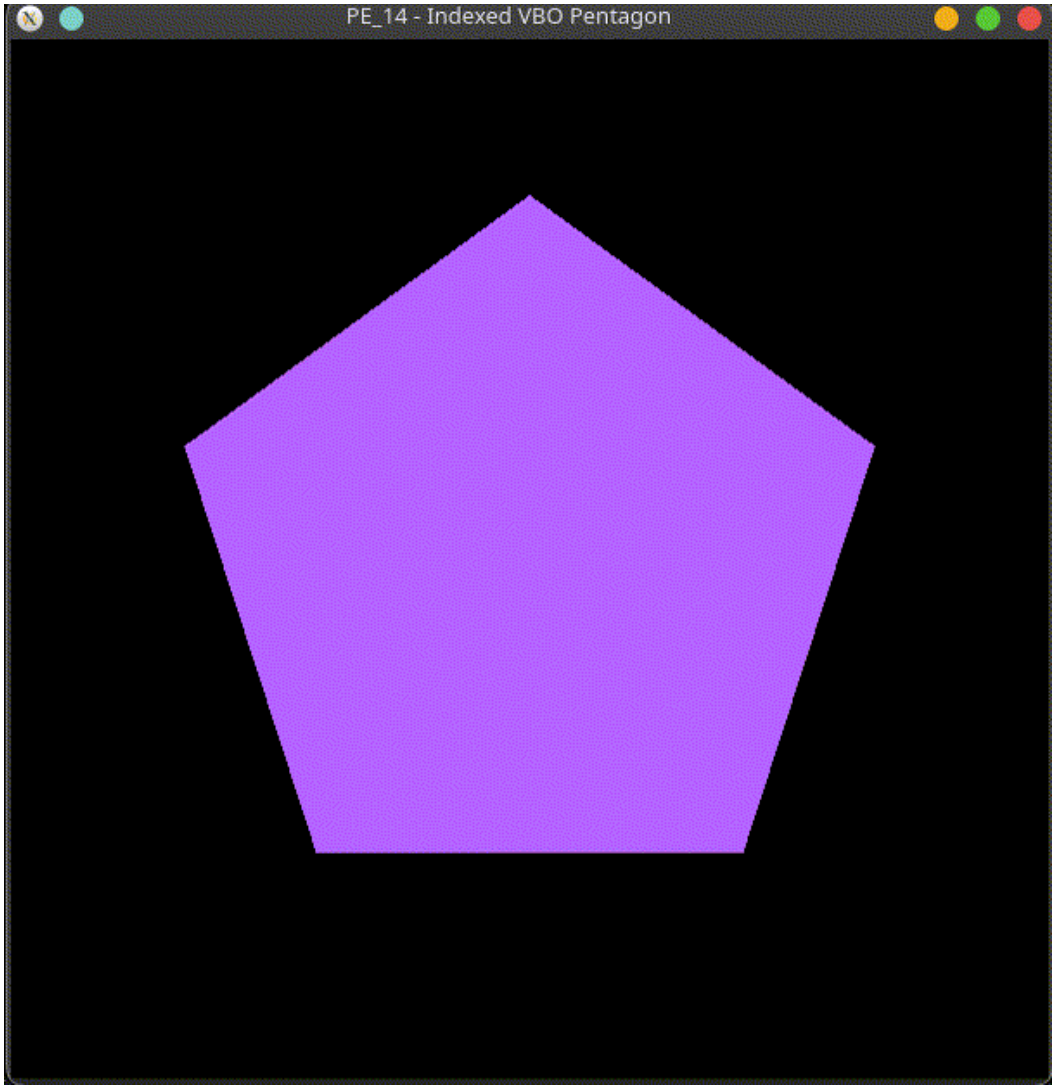
Exercise Q14 — Indexed GL_TRIANGLE_FAN Pentagon [Medium]

Objective: Practice glDrawElements with GL_TRIANGLE_FAN and a shared center vertex, entirely on the GPU.

Task (Question): Build a pentagon from GL_TRIANGLE_FAN using glDrawElements with GPU-resident position and index buffers, where the shared center vertex is stored only ONCE.

Hints:

- A triangle fan's first vertex is the shared center — every triangle in the fan reuses it.
- Your index buffer should list the center index first, then the outer ring vertices in order, closing back to the first outer vertex.



Hard Exercises

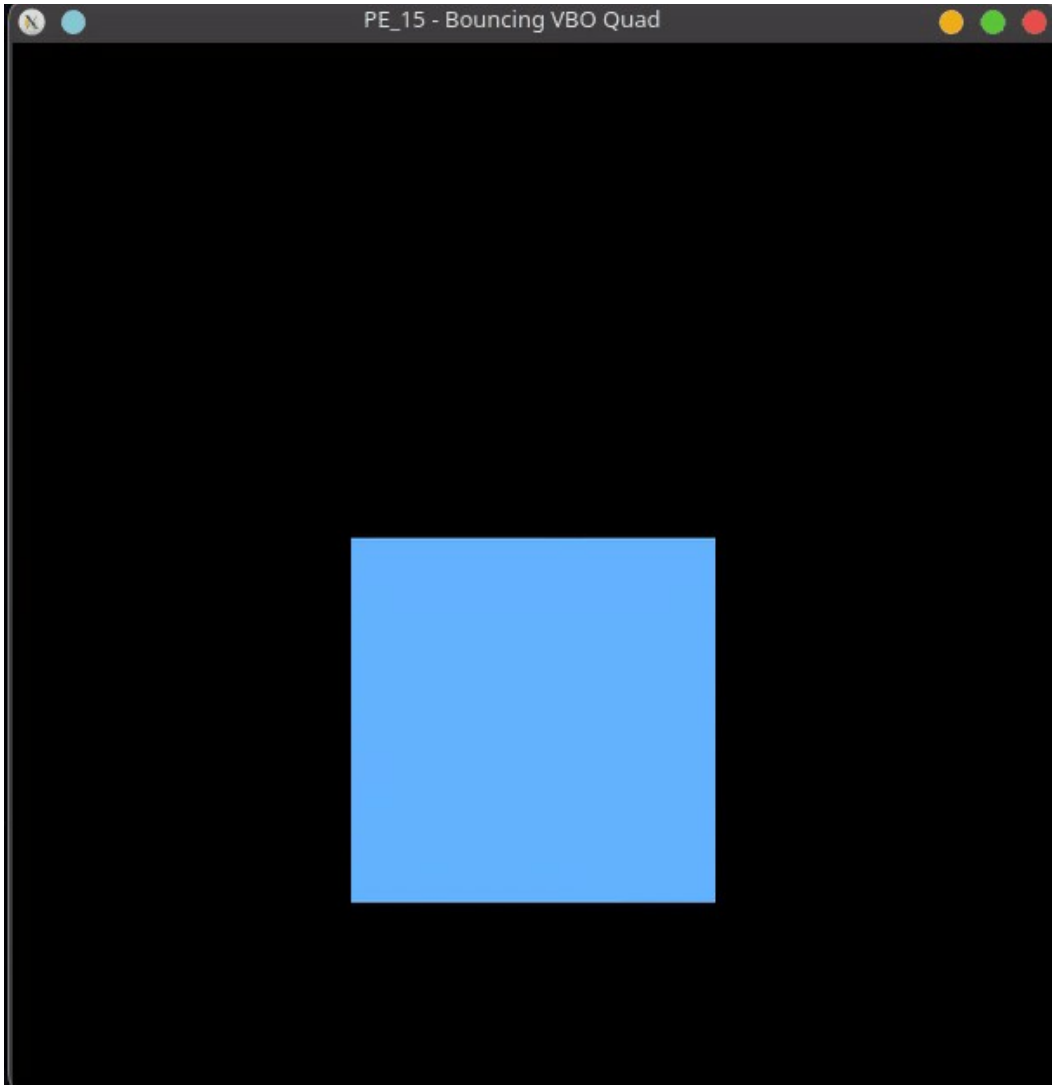
Exercise Q15 — Continuous Bouncing glMapBuffer Animation [Hard]

Objective: Practice a self-sustaining glutTimerFunc + glMapBuffer animation loop with direction reversal.

Task (Question): Animate a quad's height so it bounces up and down continuously (reversing direction at top and bottom limits), using glutTimerFunc to repeatedly map, edit, and unmap its VBO.

Hints:

- Track the current direction (up or down) and current offset in global variables outside display().
- Reverse the direction when the offset passes a chosen minimum or maximum value, then continue animating.



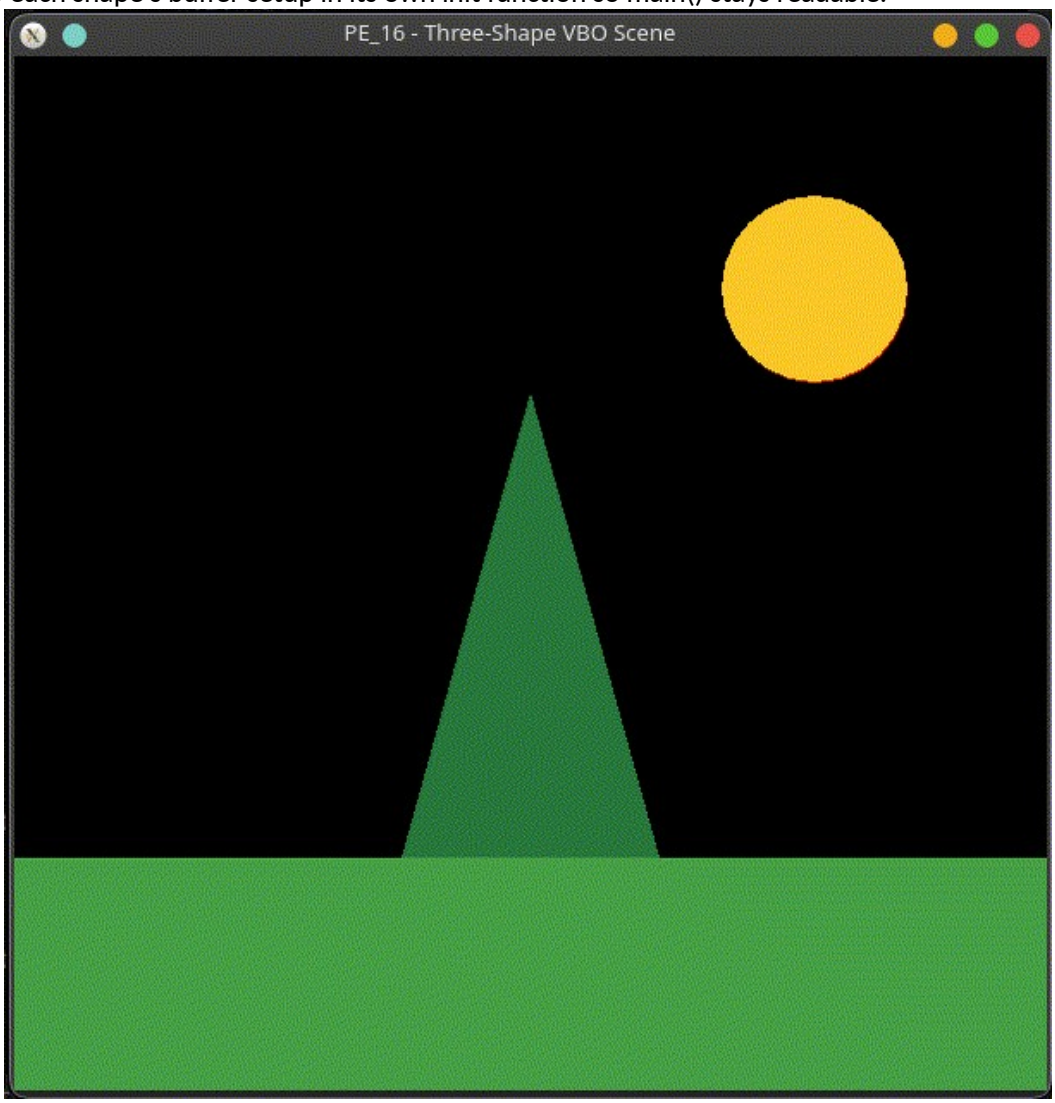
Exercise Q16 — Three-Shape VBO Scene: Sun, Indexed Ground, Tree [Hard]

Objective: Practice a multi-shape scene mixing glDrawArrays and glDrawElements across independent VBOs.

Task (Question): Build a scene with at least 3 shapes (e.g., a sun, a ground rectangle, and a tree) where EVERY shape is sourced from a VBO, mixing glDrawArrays and glDrawElements as appropriate, and the ground uses an indexed element buffer.

Hints:

- A sun or star shape is a natural fit for glDrawElements with a shared center vertex.
- Keep each shape's buffer setup in its own init function so main() stays readable.



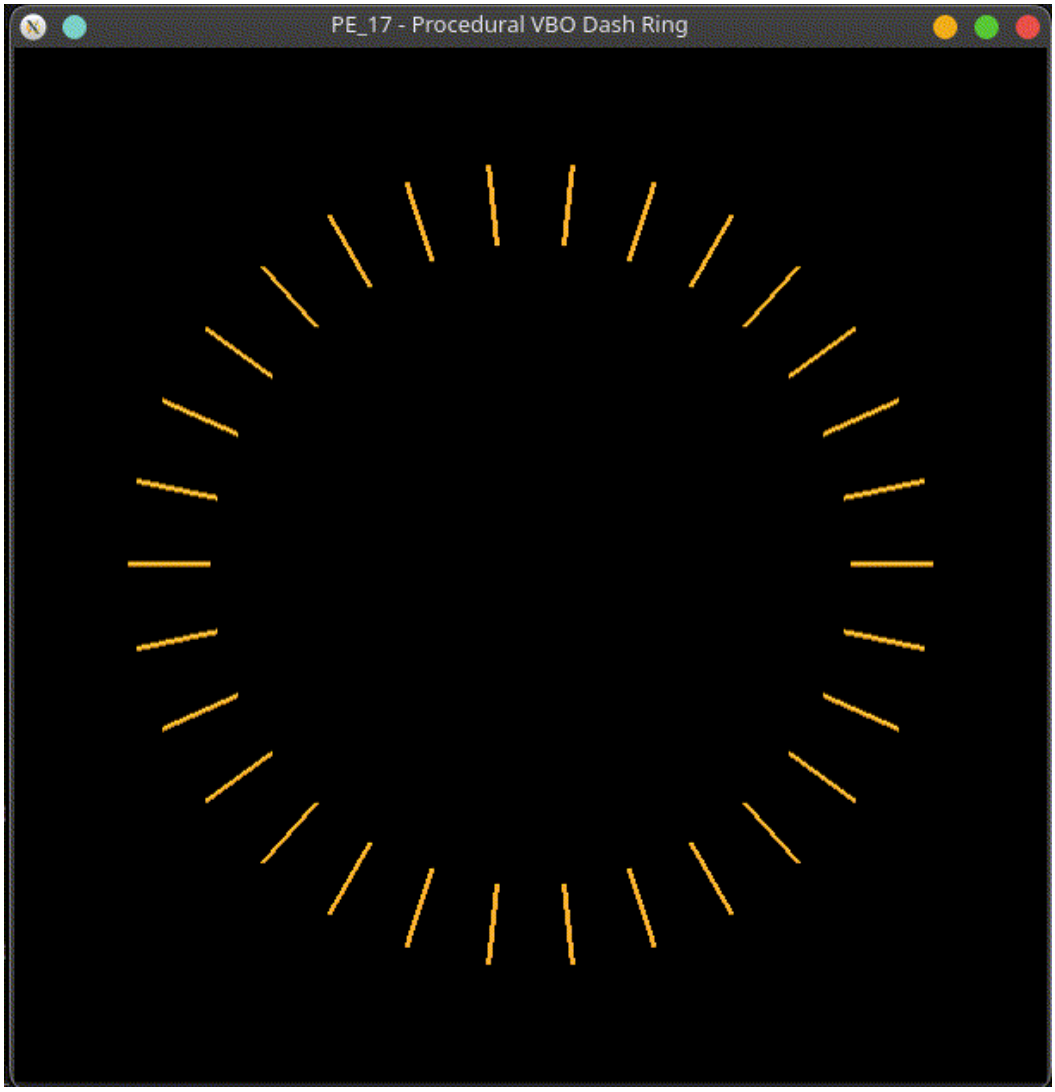
Exercise Q17 — Procedural 30-Dash Ring in One VBO [Hard]

Objective: Practice generating a larger batch of primitives procedurally and uploading them all into ONE buffer.

Task (Question): Generate 30 short dash segments arranged in a ring (using sinf/cosf in a loop) and upload them all into a SINGLE VBO, rendered with one GL_LINES glDrawArrays call.

Hints:

- Compute each dash's two endpoints from an angle that increases by (360 / 30) degrees per iteration.
- Allocate your GLfloat array BEFORE the loop, sized for 30 dashes x 2 points x however many floats per point you use.



Exercise Q18 — Interleaved + Indexed Hexagon [Hard]

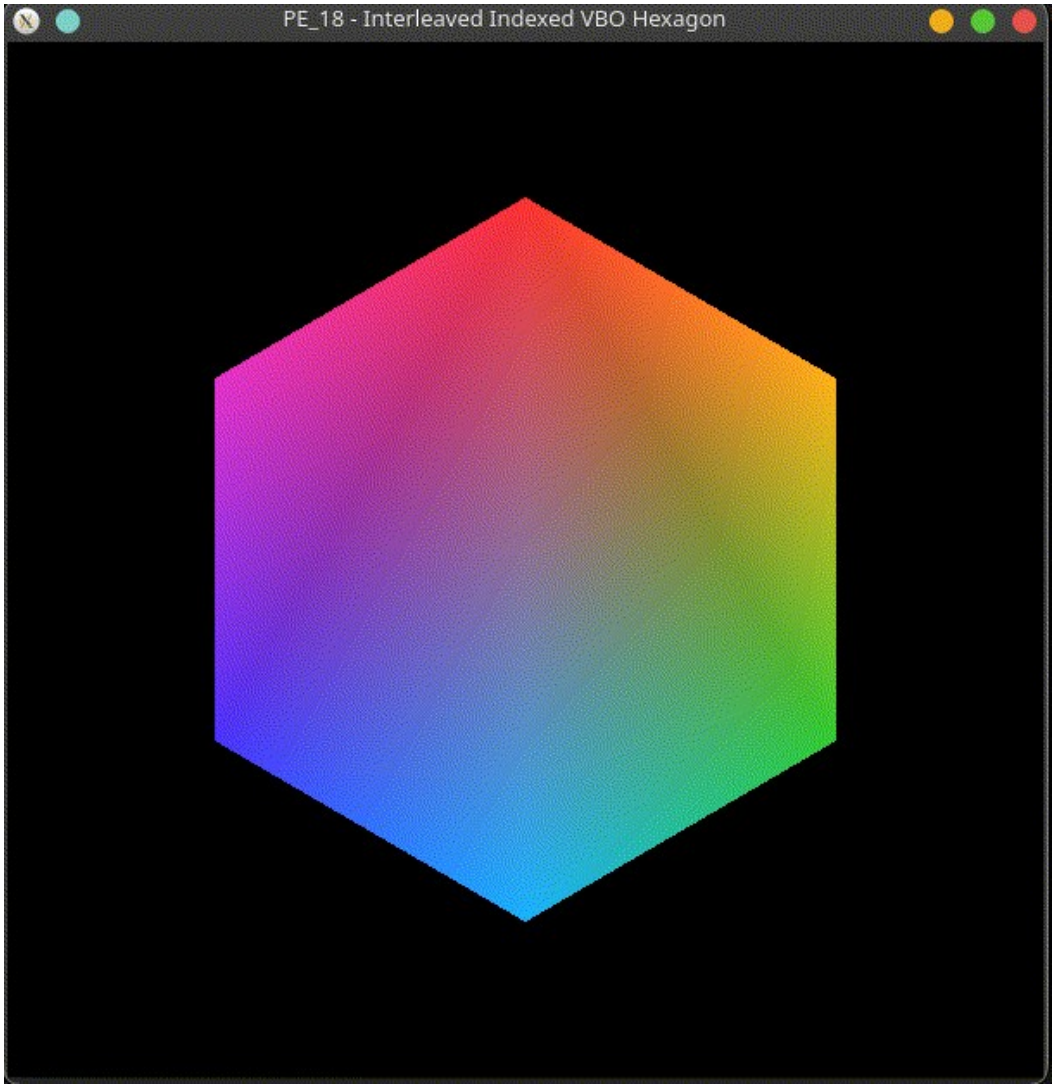
Objective: Practice combining EVERY VBO technique from this module in a single shape.

Task (Question): Build an interleaved position+color VBO for a hexagon rendered with

glDrawElements AND a non-zero stride, using a separate GPU-resident index buffer.

Hints:

- Six vertices, no shared/duplicate positions needed here — the index buffer can simply be {0,1,2,3,4,5}.
- Double-check your stride math: with 6 floats per vertex, stride = 6 * sizeof(GLfloat), and the color pointer starts 3 floats after the vertex pointer.



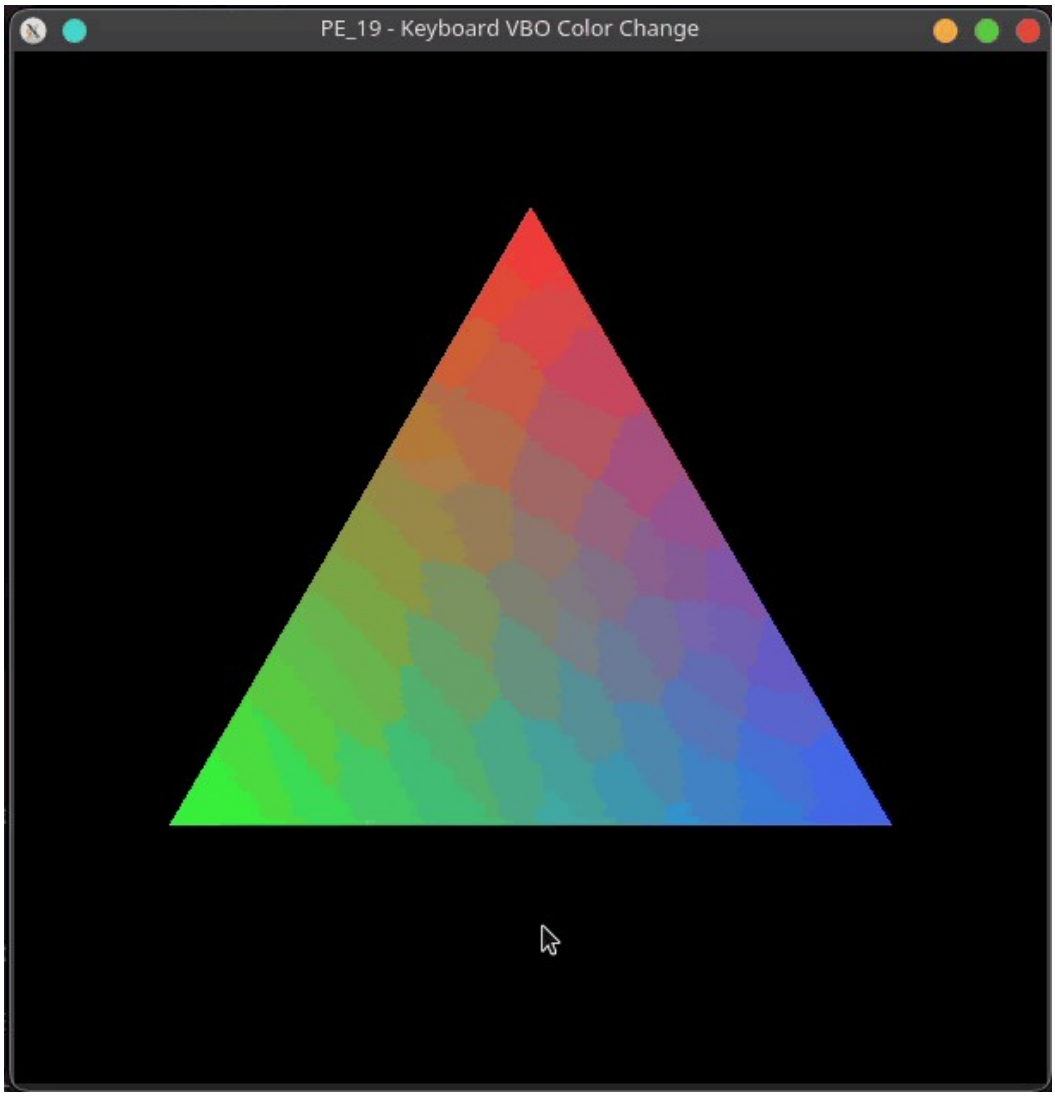
Exercise Q19 — Keyboard-Triggered Color Change via glMapBuffer [Hard]

Objective: Practice using glMapBuffer in response to a keyboard event rather than a timer.

Task (Question): Draw a shape from a position VBO and a color VBO, and use glutKeyboardFunc so pressing the 'c' key maps the COLOR buffer, cycles every vertex to a new color, and unmaps it, updating the display immediately.

Hints:

- Call glutPostRedisplay() right after unmapping so the color change is visible without waiting for another event.
- Keep a global variable tracking which color scheme is currently active, and cycle it each time 'c' is pressed.



Exercise Q20 — Capstone: Animated Ship Over Indexed Ground [Hard]

Objective: Practice combining animation, indexing, multiple buffers, and cleanup into one complete program.

Task (Question): Build a capstone scene: an animated 'ship' triangle (position+color VBO, moved every second via glutTimerFunc + glMapBuffer), a static indexed ground quad (position VBO + index VBO), and an ESC-key handler that releases all four buffers before exit(0).

Hints:

- Keep the ship's animation state (e.g., current x offset) in a global variable updated inside the timer callback.
- Count your buffers carefully before writing the cleanup handler — this capstone uses FOUR separate GLuint IDs in total.

